Lab 7 – RD Parser for Declaration Statement

Parser.c:

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "token.h"

void start_parsing();
void parse_vars();
void define_type();
void list_ids();
void handle_assignment();

struct token lookahead;

void initLexer(const char *path);
int getNextToken(struct token *t);

void raise_error(const char *was_expected)
{
    printf("Parsing Error [Line %d, Col %d]\n", lookahead.row, lookahead.col);
    printf("Missing: %s\n", was_expected);
    printf("Received: %s\n", lookahead.token_name);
    exit(1);
}

void consume(const char *target)
{
    if (strcmp(lookahead.token_name, target) == 0 ||
        strcmp(lookahead.type, target) == 0)
    {
        if (!getNextToken(&lookahead))
            strcpy(lookahead.token_name, "EOF");
    }
    else
    {
        raise_error(target);
    }
}

void start_parsing()
```

```
{
    if (strcmp(lookahead.token_name, "main") == 0)
        consume("main");
    else
        raise_error("main keyword");

    consume("(");
    consume(")");
    consume("{");

    parse_vars();
    handle_assignment();
    consume("}");

    printf("Analysis Complete: Syntax is Valid\n");
}

void parse_vars()
{
    if (strcmp(lookahead.token_name, "int") == 0 ||
        strcmp(lookahead.token_name, "char") == 0)
    {
        define_type();
        list_ids();
        consume(";");
        parse_vars();
    }
}

void define_type()
{
    if (strcmp(lookahead.token_name, "int") == 0)
        consume("int");
    else if (strcmp(lookahead.token_name, "char") == 0)
        consume("char");
    else
        raise_error("data type (int/char)");
}

void list_ids()
{
    if (strcmp(lookahead.type, "id") == 0)
    {
```

```
        consume("id");

        if (strcmp(lookahead.token_name, ",") == 0)
        {
            consume(",");
            list_ids();
        }
    }
    else
    {
        raise_error("valid identifier");
    }
}

void handle_assignment()
{
    if (strcmp(lookahead.type, "id") == 0)
    {
        consume("id");
        consume("=");

        if (strcmp(lookahead.type, "id") == 0)
            consume("id");
        else if (strcmp(lookahead.type, "num") == 0)
            consume("num");
        else
            raise_error("id or constant");

        consume(";");
    }
    else
    {
        raise_error("identifier for assignment");
    }
}

int main()
{
    initLexer("test.input");

    if (!getNextToken(&lookahead))
    {
        printf("Source file is empty.\n");
```

```
        return 0;
    }

    start_parsing();
    return 0;
}
```

Token.c:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "token.h"

#define BUFFER_SIZE 100

static const char *reserved_words[] = {
    "int", "float", "char", "if", "else",
    "for", "while", "return", "void", "switch"
};

#define TOTAL_RESERVED (sizeof(reserved_words) / sizeof(reserved_words[0]))

static FILE *input_stream = NULL;
static unsigned int current_row = 1, current_col = 0;
static int lookahead_char = 0;

void initLexer(const char *path)
{
    input_stream = fopen(path, "r");
    if (!input_stream)
    {
        perror("File Open Error");
        exit(1);
    }

    lookahead_char = fgetc(input_stream);
    current_row = 1;
    current_col = 1;
}

static int advance_cursor()
{
```

```
    if (lookahead_char == EOF)
        return EOF;

    int previous = lookahead_char;
    lookahead_char = fgetc(input_stream);

    if (previous == '\n')
    {
        current_row++;
        current_col = 1;
    }
    else
    {
        current_col++;
    }

    return previous;
}

static void clear_whitespace()
{
    while (isspace(lookahead_char))
        advance_cursor();
}

static void ignore_line()
{
    while (lookahead_char != '\n' && lookahead_char != EOF)
        advance_cursor();
}

static void ignore_block_comment()
{
    advance_cursor();
    while (lookahead_char != EOF)
    {
        if (lookahead_char == '*')
        {
            advance_cursor();
            if (lookahead_char == '/')
            {
                advance_cursor();
                break;
```

```
            }
        }
        else
        {
            advance_cursor();
        }
    }
}

static void capture_word(char *buffer)
{
    int idx = 0;
    while (isalnum(lookahead_char) || lookahead_char == '_')
    {
        if (idx < BUFFER_SIZE - 1)
            buffer[idx++] = advance_cursor();
        else
            advance_cursor();
    }
    buffer[idx] = '\0';
}

static void capture_operator(char *buffer)
{
    int idx = 0;
    buffer[idx++] = advance_cursor();

    char next = lookahead_char;
    char first = buffer[0];

    if ((first == '+' && next == '+') || (first == '-' && next == '-') ||
        (first == '=' && next == '=') || (first == '!' && next == '=') ||
        (first == '<' && next == '=') || (first == '>' && next == '=') ||
        (first == '&' && next == '&') || (first == '|' && next == '|'))
    {
        buffer[idx++] = advance_cursor();
    }

    buffer[idx] = '\0';
}

int getNextToken(struct token *tok)
{
```

```
    char text[BUFFER_SIZE];

    while (lookahead_char != EOF)
    {
        clear_whitespace();

        int active_char = lookahead_char;
        if (active_char == EOF)
            return 0;

        if (active_char == '#') {
            ignore_line();
            if (lookahead_char == '\n')
                advance_cursor();
            continue;
        }

        if (active_char == '/')
        {
            advance_cursor();
            if (lookahead_char == '/')
            {
                ignore_line();
                continue;
            }
            else if (lookahead_char == '*')
            {
                ignore_block_comment();
                continue;
            }
            else
            {
                text[0] = '/';
                text[1] = '\0';
                tok->row = current_row;
                tok->col = current_col - 1;
                strcpy(tok->token_name, text);
                idTokenType(tok);
                return 1;
            }
        }

        if (strchr("+-*%=<>!&|", active_char))
```

```
        {
            tok->row = current_row;
            tok->col = current_col;
            capture_operator(text);
            strcpy(tok->token_name, text);
            idTokenType(tok);
            return 1;
        }

        if (isSpecSym(active_char))
        {
            tok->row = current_row;
            tok->col = current_col;
            text[0] = advance_cursor();
            text[1] = '\0';
            strcpy(tok->token_name, text);
            idTokenType(tok);
            return 1;
        }

        if (isalnum(active_char) || active_char == '_')
        {
            tok->row = current_row;
            tok->col = current_col;
            capture_word(text);
            strcpy(tok->token_name, text);
            idTokenType(tok);
            return 1;
        }

        advance_cursor();
    }

    return 0;
}

int isArithOp(const char *str)
{
    return (!strcmp(str, "+") || !strcmp(str, "-") || !strcmp(str, "*") ||
            !strcmp(str, "/") || !strcmp(str, "%") || !strcmp(str, "++") ||
            !strcmp(str, "--"));
}
```

```c
int isRelOp(const char *str)
{
    return (!strcmp(str, "<") || !strcmp(str, ">") || !strcmp(str, "<=") ||
            !strcmp(str, ">=") || !strcmp(str, "==") || !strcmp(str, "!=") ||
            !strcmp(str, "="));
}

int isLogOp(const char *str)
{
    return (!strcmp(str, "&&") || !strcmp(str, "||") || !strcmp(str, "!"));
}

int isNum(const char *str)
{
    int i = 0, points = 0;
    if (str[i] == '+' || str[i] == '-') i++;
    for (; str[i]; i++) {
        if (str[i] == '.') {
            points++;
            if (points > 1) return 0;
        } else if (!isdigit(str[i])) return 0;
    }
    return (i > 0);
}

int isStringLit(const char *str)
{
    size_t len = strlen(str);
    return (len >= 2 && str[0] == '"' && str[len - 1] == '"');
}

int isKey(const char *str)
{
    for (int i = 0; i < TOTAL_RESERVED; i++)
        if (strcmp(str, reserved_words[i]) == 0) return 1;
    return 0;
}

int isIden(const char *str)
{
    if (!isalpha(str[0]) && str[0] != '_') return 0;
    for (int i = 1; str[i]; i++)
        if (!isalnum(str[i]) && str[i] != '_') return 0;
```

```
    return !isKey(str);
}

int isSpecSym(char c)
{
    return (c == ';' || c == ',' || c == '(' || c == ')' ||
            c == '{' || c == '}' || c == '[' || c == ']' ||
            c == ':' || c == '?');
}

void idTokenType(struct token *item)
{
    const char *name = item->token_name;
    if (isKey(name)) strcpy(item->type, "kw");
    else if (isArithOp(name)) strcpy(item->type, "arithop");
    else if (isRelOp(name)) strcpy(item->type, "relop");
    else if (isLogOp(name)) strcpy(item->type, "logop");
    else if (isNum(name)) strcpy(item->type, "num");
    else if (isStringLit(name)) strcpy(item->type, "literal");
    else if (isIden(name)) strcpy(item->type, "id");
    else strcpy(item->type, "specsym");
}
```

Token.h:

```
#ifndef TOKEN_H
#define TOKEN_H

#define TOKEN_MAX_LEN 100

struct token
{
    char token_name[TOKEN_MAX_LEN];
    int index;
    unsigned int row, col;
    char type[TOKEN_MAX_LEN];
};

void initLexer(const char *path);
int getNextToken(struct token *tok);

int isArithOp(const char *str);
int isRelOp(const char *str);
int isLogOp(const char *str);
```

```
int isNum(const char *str);
int isStringLit(const char *str);
int isKey(const char *str);
int isIden(const char *str);
int isSpecSym(char c);
void idTokenType(struct token *item);

#endif
```

Output with input.c file:

```
6CSEC1@dbl-23:~/Documents/CD_230905152/L7$ gcc parser.c token.c -o abc
6CSEC1@dbl-23:~/Documents/CD_230905152/L7$ ./abc
Parsing Successful
6CSEC1@dbl-23:~/Documents/CD_230905152/L7$ cat input.c
#include<stdio.h>
main(){
    int a,b,c;
    //Hi World
    b=10;
}6CSEC1@dbl-23:~/Documents/CD_230905152/L7$ ./abc
Syntax Error at line 5, column 5
Expected: ;
Found: b
6CSEC1@dbl-23:~/Documents/CD_230905152/L7$ cat input.c
#include<stdio.h>
main(){
    int a,b,c
    //Hi World
    b=10;
}6CSEC1@dbl-23:~/Documents/CD_230905152/L7$ ./abc
Syntax Error at line 5, column 5
Expected: =
Found: b
6CSEC1@dbl-23:~/Documents/CD_230905152/L7$ cat input.c
#include<stdio.h>
main(){
    int a,b,c;qwerty
    //Hi World
    b=10;
```